

FATEC JACAREÍ

Faculdade de Tecnologia de Jacareí

Error Squad

Felipe Pacheco | Leonardo Irineu | Carlos Eduardo
João Vitor | Arthur Facchinetti | Caio Araujo

**MODELAGEM MONGODB E
EMBEDDING VS REFERENCING**

Dia 2 – Modelagem MongoDB e Embedding vs Referencing

Jacareí – SP

2026

SUMÁRIO

1	Objetivo da Modelagem	3
2	Visão C4 – Contexto	3
3	Visão C4 – Container	3
4	Coleções do Modelo	4
4.1	Responsabilidade de Cada Coleção	4
5	Visão C4 – Componentes de Dados	5
6	Embedding vs Referencing	6
6.1	Onde Usamos Embedding	6
6.2	Onde Usamos Referencing	7
7	Diagrama de Decisão da Modelagem	8
8	Modelo ER Orientado a MongoDB	8
9	Fluxo Principal com MongoDB	9
10	Regras Atendidas pela Modelagem	9
11	Índices Recomendados	10
12	Conclusão	10

1 OBJETIVO DA MODELAGEM

A modelagem MongoDB do sistema de gestão de leads da **1000 Valle Multimarcas** foi definida para apoiar o ciclo comercial completo:

- cadastro e qualificação de leads;
- vínculo do lead com cliente, loja e atendente;
- acompanhamento da negociação ativa;
- registro do histórico da negociação;
- auditoria das ações realizadas;
- consultas operacionais e indicadores gerenciais.

O desenho prioriza consultas frequentes do funil comercial, rastreabilidade das operações e redução de redundância entre entidades reutilizadas.

2 VISÃO C4 – CONTEXTO

A visão de contexto apresenta os atores externos que interagem com o sistema e seus respectivos papéis no fluxo de gestão de leads.

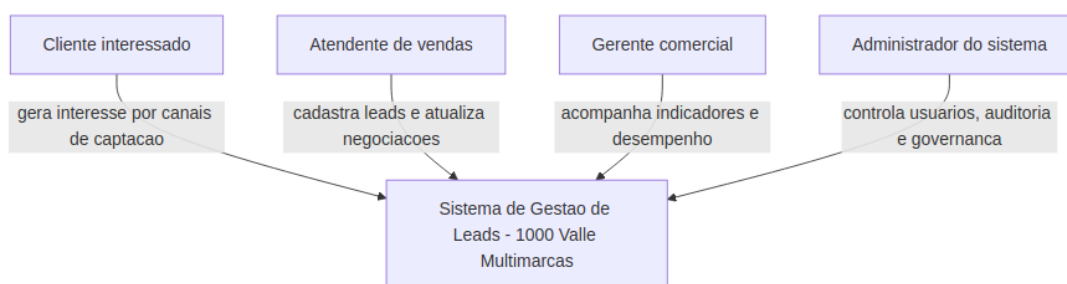


Figura 1 – Visão C4 de contexto: atores e sistema.

Fonte: elaborado pelos autores (2026).

3 VISÃO C4 – CONTAINER

A visão de container detalha os componentes tecnológicos do sistema, incluindo os canais de captação, a camada de aplicação, o banco de dados MongoDB e o dashboard gerencial.

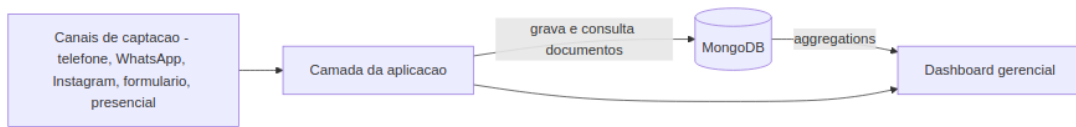


Figura 2 – Visão C4 de container: componentes tecnológicos.

Fonte: elaborado pelos autores (2026).

4 COLEÇÕES DO MODELO

As coleções principais permanecem as mesmas definidas no Dia 1:

1. clientes
2. leads
3. usuarios
4. negociacoes
5. logs
6. lojas

4.1 Responsabilidade de Cada Coleção

clientes

Armazena os dados cadastrais do cliente interessado.

Campos principais:

- `_id`
- `nome`
- `telefone`
- `email`

leads

Representa a oportunidade comercial gerada a partir do interesse de um cliente.

Campos principais:

- `_id`
- `clienteId` (ref. `clientes._id`)
- `atendenteId` (ref. `usuarios._id`)
- `lojaId` (ref. `lojas._id`)

- origem
- status
- prioridade
- createdAt

usuarios

Representa atendentes, gerentes e administradores do sistema.

Campos principais:

- _id
- nome
- email
- perfil
- ativo

negociacoes

Registra o processo comercial de um lead.

Campos principais:

- _id
- leadId (ref. leads._id)
- ativa
- estagioAtual
- historico (eventos embutidos)
- valorProposto

logs

Registra a trilha de auditoria das operações realizadas.

Campos principais:

- _id
- leadId (ref. leads._id)
- usuarioId (ref. usuarios._id)
- acao
- detalhes

- createdAt

lojas

Representa as unidades da 1000 Valle Multimarcas.

Campos principais:

- _id
- nome
- cidade
- estado
- ativo

5 VISÃO C4 – COMPONENTES DE DADOS

O diagrama abaixo ilustra as coleções MongoDB e seus relacionamentos por referência.

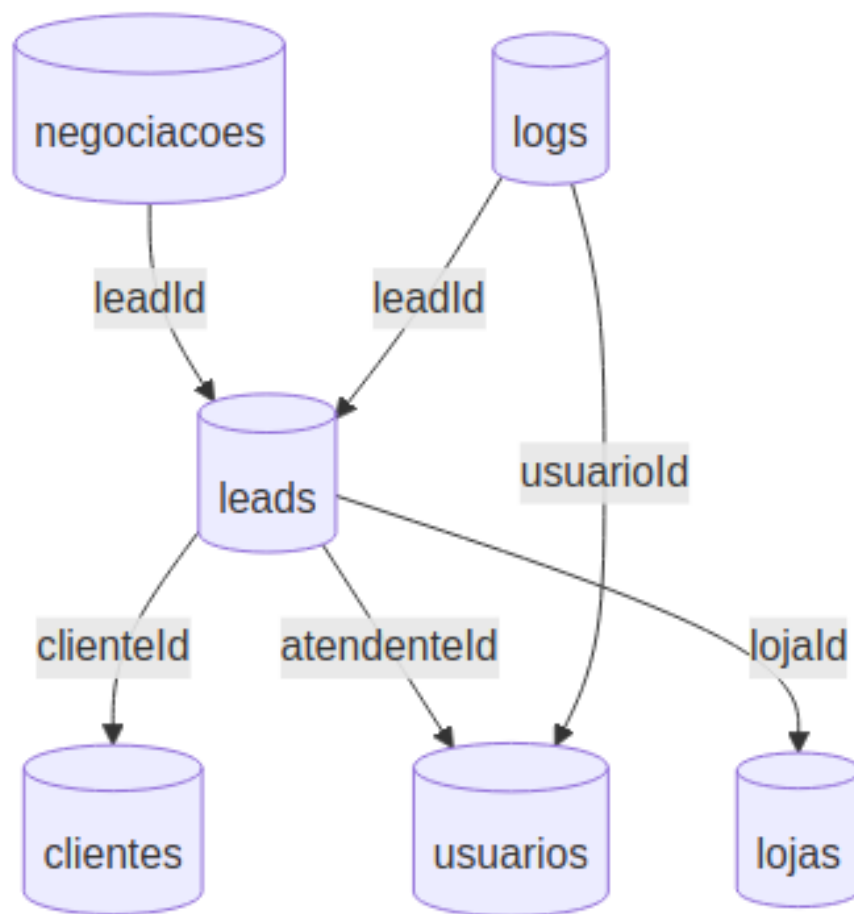


Figura 3 – Visão C4 de componentes: coleções e referências.
 Fonte: elaborado pelos autores (2026).

6 EMBEDDING VS REFERENCING

No MongoDB, a decisão entre embutir documentos ou referenciar coleções depende do padrão de leitura, crescimento dos dados e independência das entidades.

6.1 Onde Usamos Embedding

Usamos *embedding* em `negociacoes.historico`. Exemplo de estrutura:

```

{
  _id: ObjectId("..."),
  leadId: ObjectId("..."),
  ativa: true,
  estagioAtual: "proposta_enviada",
  historico: [
    {

```

```

    estagio: "contato_inicial",
    status: "em_atendimento",
    observacao: "Cliente demonstrou interesse em SUV.",
    responsavelId: ObjectId("..."),
    createdAt: ISODate("2026-05-11T10:00:00Z")
  },
  {
    estagio: "proposta_enviada",
    status: "em_negociacao",
    observacao: "Proposta enviada pelo atendente.",
    responsavelId: ObjectId("..."),
    createdAt: ISODate("2026-05-11T11:30:00Z")
  }
]
}

```

Motivos:

- o histórico pertence diretamente a uma negociação;
- os eventos do histórico são consultados junto com a negociação;
- a leitura fica mais simples e rápida;
- evita consultas extras para montar a linha do tempo da negociação.

6.2 Onde Usamos Referencing

Usamos *referencing* nos relacionamentos entre entidades que podem ser reutilizadas, alteradas de forma independente ou crescer em coleções separadas.

Referências definidas:

- leads.clienteId -> clientes._id
- leads.atendenteId -> usuarios._id
- leads.lojaId -> lojas._id
- negociacoes.leadId -> leads._id
- logs.leadId -> leads._id
- logs.usuarioId -> usuarios._id

Motivos:

- um cliente pode gerar mais de um lead ao longo do tempo;
- um atendente pode cuidar de muitos leads;
- uma loja recebe muitos leads;
- logs podem crescer bastante e devem ficar separados;
- evita duplicação de dados cadastrais;

- reduz risco de inconsistências quando clientes, usuários ou lojas forem atualizados.

7 DIAGRAMA DE DECISÃO DA MODELAGEM

O fluxograma a seguir sintetiza o processo de decisão entre *embedding* e *referencing* adotado na modelagem.

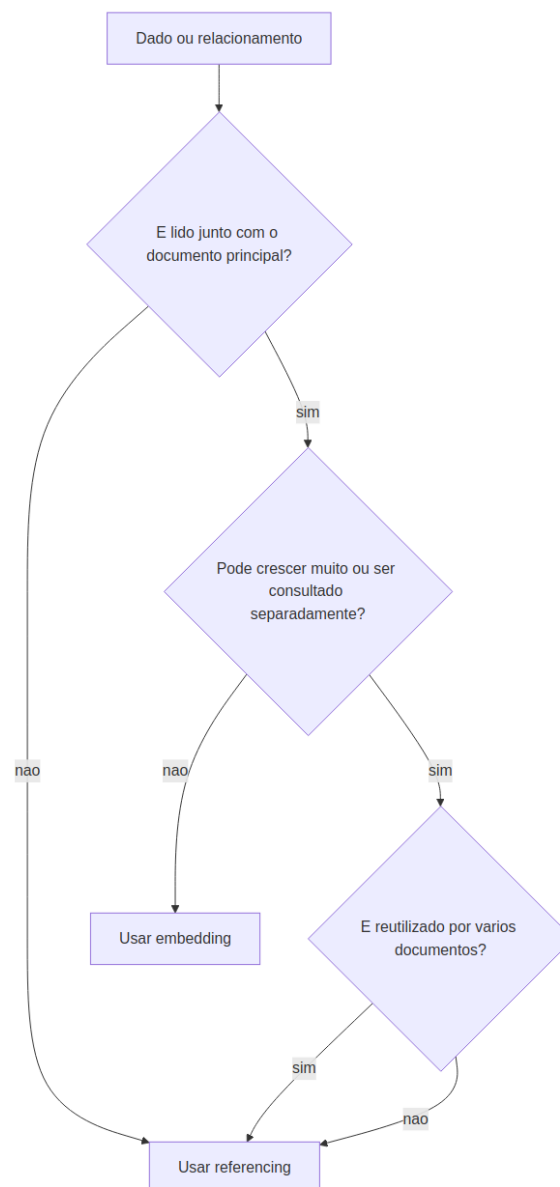


Figura 4 – Diagrama de decisão: *embedding* vs *referencing*.
Fonte: elaborado pelos autores (2026).

8 MODELO ER ORIENTADO A MONGODB

O modelo entidade-relacionamento apresenta todas as coleções com seus atributos e as cardinalidades dos relacionamentos, seguindo a abordagem orientada a documentos do MongoDB.

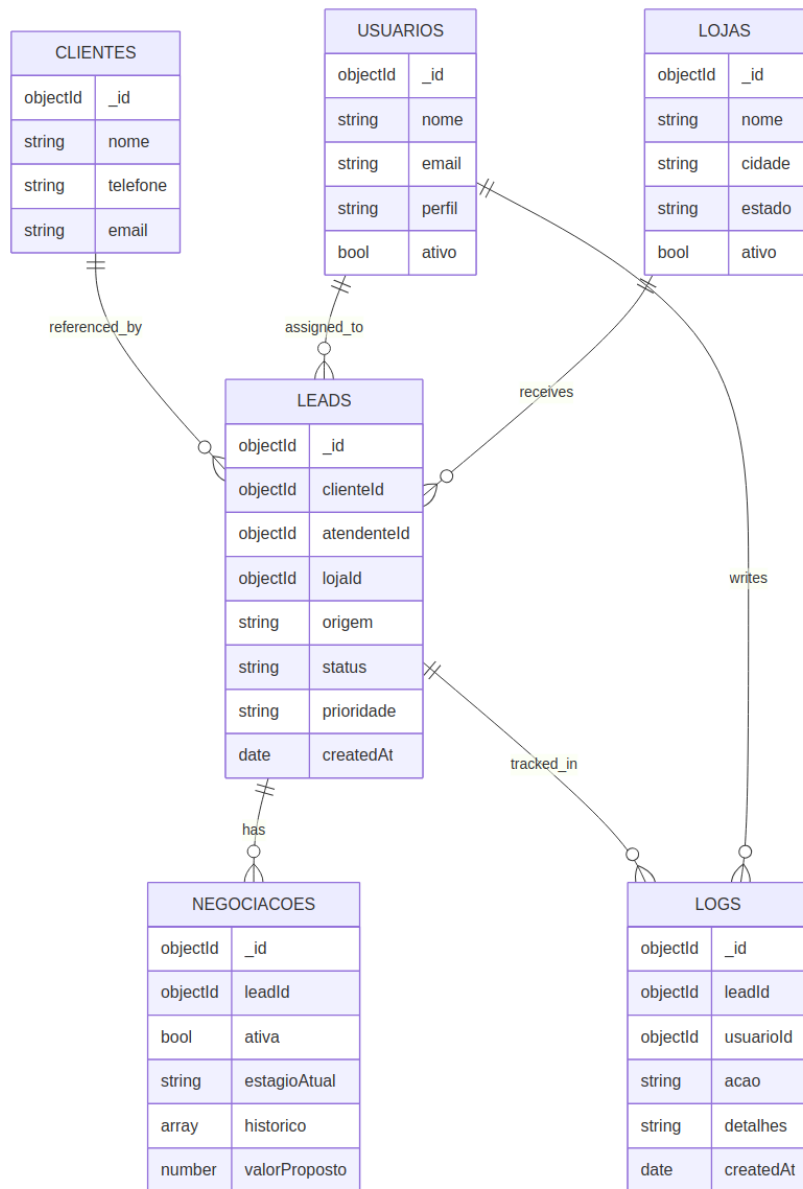


Figura 5 – Modelo ER completo orientado a MongoDB.

Fonte: elaborado pelos autores (2026).

9 FLUXO PRINCIPAL COM MONGODB

O diagrama de sequência a seguir representa o fluxo operacional completo, desde o interesse do cliente até o registro de auditoria no MongoDB.

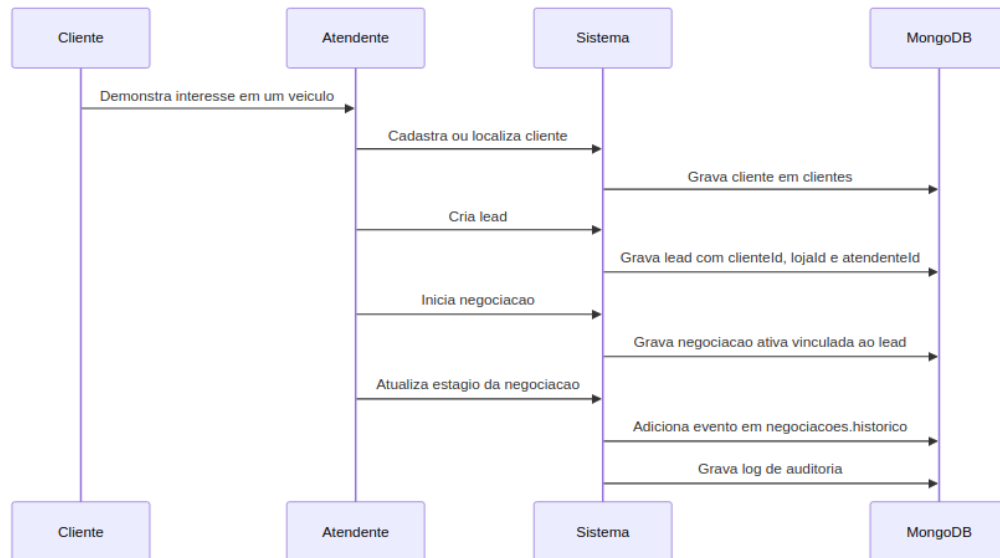


Figura 6 – Diagrama de sequência do fluxo principal com MongoDB.

Fonte: elaborado pelos autores (2026).

10 REGRAS ATENDIDAS PELA MODELAGEM

- Cada lead fica vinculado a um cliente por `clienteId`.
- Cada lead fica vinculado a uma loja por `lojaId`.
- Cada lead fica vinculado a um atendente por `atendenteId`.
- Cada negociação fica vinculada a um lead por `leadId`.
- O histórico da negociação fica registrado por embedding em `negociacoes.historico`.
- Os logs ficam separados para auditoria e crescimento independente.
- A regra de apenas uma negociação ativa por lead deve ser garantida pela aplicação e por índice único parcial.

11 ÍNDICES RECOMENDADOS

Os índices abaixo foram definidos para otimizar as consultas operacionais mais frequentes e garantir a unicidade da negociação ativa por lead:

```
db.leads.createIndex({ clienteId: 1 });
```

```
db.leads.createIndex({ atendenteId: 1 });
db.leads.createIndex({ lojaId: 1 });
db.leads.createIndex({ status: 1, createdAt: -1 });
db.negociacoes.createIndex(
  { leadId: 1, ativa: 1 },
  { unique: true, partialFilterExpression: { ativa: true } }
);
db.logs.createIndex({ leadId: 1, createdAt: -1 });
db.logs.createIndex({ usuarioId: 1, createdAt: -1 });
```

12 CONCLUSÃO

A modelagem combina *referencing* para entidades principais e *embedding* para o histórico interno da negociação. Essa decisão atende ao fluxo de vendas da **1000 Valle Multimarcas** porque mantém os dados cadastrais normalizados, reduz duplicidade e permite consultar a negociação com sua linha do tempo em um único documento.